

EEE4120F Practical 4

1st Ebrahim Bhyat
BHYEBR002

2nd Joshua Smith
SMTJOS022

I. INTRODUCTION

Predictable, deterministic execution is a fundamental requirement for microprocessors deployed in resource-constrained embedded systems. The OrbitEdge-1 mission demands a low-power, radiation-tolerant edge-computing core capable of processing telemetry and sensor data autonomously, without the complexity or power overhead associated with multi-stage pipelined architectures. This report documents the design, implementation, and functional verification of the StarCore-1 microprocessor — the primary computational element of the OrbitEdge-1 flight management computer.

Rather than implementing the processor in sequential software, Verilog Hardware Description Language (HDL) was used to describe the hardware at the Register-Transfer Level (RTL). A single-cycle execution model was adopted so that every instruction, from fetch through to write-back, completes within one clock period. A Harvard architecture was selected as the structural basis of the design: by maintaining physically separate instruction (ROM) and data (RAM) memory buses, the Von Neumann bottleneck is eliminated and simultaneous instruction fetch and data access are supported without arbitration logic.

The implementation followed a modular decomposition strategy in which individual hardware blocks were designed and verified independently before system integration. Core datapath elements (the Arithmetic Logic Unit (ALU), General-Purpose Register file (GPR), and dual memories) were implemented first. The ALU Control Unit and Control Unit were subsequently developed to generate and route the correct control signals across the datapath. Each module was validated through dedicated testbenches before being integrated into the top-level StarCore1 wrapper module.

II. ARCHITECTURE OVERVIEW

The StarCore-1 is a 16-bit, RISC processor relying on a Harvard architecture as mentioned above, and in doing so the instruction (ROM) and data (RAM) memory buses prevent bottlenecks due to a unified memory as seen in the Von Neumann architecture. This allows for simultaneous fetching of instructions while performing data access operations. This allows every instruction to be completed in one cycle. Due to power efficiency being critical, the design is suitable due to how simplistic it is.

Three instruction encodings are defined across 16-bit words, summarised in Figure 1. R-type instructions place the opcode in bits [15:12], source register RS1 in [11:9], source register

RS2 in [8:6], and the write-back destination WS in [5:3]. I-type instructions encode a 6-bit signed immediate offset in [5:0] with WS in [8:6]. J-type instructions dedicate bits [11:0] to a 12-bit jump target offset.

Fig. 1. Instruction Types

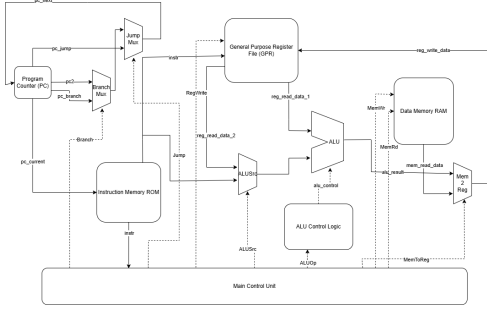
Format	Bit Layout [15 : 0]
R-type	[15:12] OP — [11:9] RS1 — [8:6] RS2 — [5:3] WS — [2:0] unused
I-type	[15:12] OP — [11:9] RS1 — [8:6] WS — [5:0] Offset (6-bit signed)
J-type	[15:12] OP — [11:0] Offset (12-bit target)

The 16-bit Program Counter (PC) is most vital to the instruction fetch logic as it manages the execution flow. In order to point to the next 16-bit instruction word, the PC increments by 2. For other instructions such as branch (BNE , BEQ) and jump (JMP), the architecture integrates a dedicated branch adder and target multiplexer. Bit slicing logic is used as instruction memory is word-addressed in contrast to the PC which is byte-addressed. In doing so, memory addresses are correctly assigned during the fetch operation.

The Control Unit and Arithmetic Logic Unit are responsible for the control and execution phases. The 4-bit opcode is decoded by the Control Unit into a control signal which determines the datapaths behavior. The General-Purpose Register contains two asynchronous read ports for source registers RS1 and RS2, as well as an asynchronous write port for the destination register. During execution, the ALU Control module defines operations which are carried out by the ALU, whilst simultaneously allowing for conditional branching through a Zero flag.

Lastly there is the write-back and data memory interaction in the datapath. The ALU computes the effective memory address through adding an offset to a base register value for load and store operations. Asynchronous read and write operations are supported by the `DataMemory.v` module. The `Mem2Reg` multiplexer determines whether data returned to the register file is from the data memory or ALU.

Fig. 2. Simplistic view of the Harvard Architecture implemented



III. VERIFICATION METHODOLOGY

A. Arithmetic Logic Unit

Verification of the ALU focuses on validating 16-bit combinational logic across eight operations while utilising automated reporting to prevent error propagation. Specific input pairs are assigned to operands *A* and *B*, with the testbench confirming the internal 'case' logic after a settling delay.

Corner-case test vectors target boundary conditions, verifying unsigned wrap-around during arithmetic, logical integrity via alternating bit patterns, and shift-amount masking. SLT logic is further validated across all comparison states. Final validation combines automatic terminal logging of mismatches with waveform inspection in GTKWave to ensure signal stability and confirm that the combinational logic remains free of unintended latches or glitches.

B. General Purpose Register

Verification of the GPR utilises a modular testbench to validate the timing distinction between synchronous write and asynchronous read operations. The methodology focuses on sequential validation of all eight registers, dual-port read concurrency, and write-enable protection.

The testbench confirms that all registers are independently addressable by applying various instructions and simultaneously exercising both read ports to ensure two operands are available to the ALU within a single cycle. Crucially, the test environment explicitly attempts to write dummy data to Register 0 (R0) to validate the hardwired zero-protection logic, confirming that the register successfully rejects the write and continuously outputs zero. Final GTKWave analysis confirms synchronous behavior, verifying that data is captured only on the rising clock edge while read ports respond immediately to address changes, ensuring stability for integration into the StarCore-1.

C. Instruction Memory

Verification focuses on the correct initialisation of the 16-word ROM and the accuracy of the combinational addressing logic. Given the Harvard architecture, the fetch path must be purely combinational to ensure instructions are available immediately after the PC settles.

The testbench compares the instruction output against a "golden" array mirroring the 'test.prog' file as the PC increments, verifying the bit-slicing logic used for 4-bit word

addressing. Functional correctness is flagged via terminal logs, while GTKWave analysis confirms that the ROM responds to PC updates instantaneously, ensuring fetch, decode, and execute stages settle within a single clock cycle.

D. Data Memory

As the primary storage for sensor data and telemetry, verification focuses on the interaction between the synchronous write port and the combinational gated read port. The testbench sequence ensures the 8-word memory array operates strictly within specification.

Initial state verification confirms the RAM populates correctly from the `test.data` file at simulation start. The testbench then executes sequential write-read cycles, driving 16-bit hexadecimal patterns to all addresses on the positive clock edge and immediately reading them back asynchronously to confirm precise indexing and data stability.

Edge-case timing analysis validates the bus control logic. De-asserting the read-enable (`MemRd`) confirms the module outputs a strict zero, preventing spurious data from entering the write-back multiplexer. Finally, disabled-write safety is verified by attempting overwrites while write-enable (`MemWr`) is low, ensuring synchronous memory protection. Results are automatically compared against expected values and analysed in GTKWave.

E. ALU Control

Verification concerns mainly the two-level decoding logic interfaced between the Control Unit with the ALU. The module itself is responsible for translation of high-level instruction operations into 3-bit operational commands. A truth table verification approach has been implemented in which every combination of the 2-bit ALU Opcode and 4-bit Opcode inputs are driven to confirm the combinational mapping.

Static and dynamic decoding priorities are used within stimulus generation to ensure total logic coverage. Address calculations and comparisons are hard-wired correctly through the module outputting an addition or subtraction command regardless of the opcode when the `ALUOp` is set to memory access or branch modes. The testbench then iterates through the R-type opcodes to verify that it correctly maps to the corresponding ALU function.

Comparison is then done through comparison of the modules 3-bit output against a pre-calculated golden measure. For each test vector, a 10ns delay is simulated to ensure settling of combinational logic. Discrepancies are monitored and logged to assist with debugging. This in combination with visual analysis done on GTKWave to ensure the purely combinational nature of the module.

F. Control Unit

Verification of the Control Unit tests the precision of the processor's command center. The testbench approach entails an exhaustive search of the opcode space to ensure that defined instructions as well as reserved and undefined states results in the exact control word in the architectural truth table.

A verification task evaluates all of the ten output signals simultaneously against a golden measure model. Emphasis is placed on the reserved opcode '1010' and undefined opcodes to ensure they do not assert any control signals, which prevents the processor from entering indeterminate states due to incorrect instruction patterns.

Validation is achieved through visual waveform analysis, where similarly to before, any mismatches are highlighted to ease the debugging process. It can then be observed and confirmed visually that the control signals transition properly once the opcode is changed.

G. Star Core

The full-system integration of the StarCore-1 processor is verified through a passive testbench that monitors hierarchical signal references as the system executes real machine-code programs in a unified environment. Utilising a program-driven stimulus approach, the methodology involves pre-loading instructions into simulated ROM and RAM, followed by a cycle-by-cycle execution trace that logs the Program Counter and general-purpose register states at every rising clock edge.

This real-time visibility allows one to validate data flow through the ALU and confirm that control dependencies from the Control Unit dynamically reconfigure the datapath without structural hazards. To ensure the processor's functional readiness for satellite flight management, signal transitions are captured in VCD files for analysis in GTKWave, where single-cycle timing constraints are verified alongside the mathematical accuracy of multi-instruction sequences and the correct handling of conditional branches.

IV. RESULTS

The verification of the StarCore-1 processor was conducted through a tiered testing strategy, progressing from individual component validation to full-system integration. By isolating sub-modules before observing the unified datapath, the testing framework successfully verified both combinatorial logic accuracy and synchronous inter-module dependencies. The following results were observed during functional simulation.

A. ALU Functional Performance

The Arithmetic Logic Unit (ALU) successfully executed its complete instruction set within the 10ns single-cycle timing constraint. The functional mapping of the 3-bit ALUcnt control signal to the corresponding arithmetic and logical outputs is confirmed in the simulation trace in Figure 3.

Boundary condition testing revealed that the ALU correctly handles unsigned wrap-around during arithmetic operations. Furthermore, logical integrity was maintained during bitwise masking and shifting operations. A critical observation from the waveform trace is the immediate settling of output buses following changes to the control signals or input operands. This confirms the purely combinational nature of the ALU path.

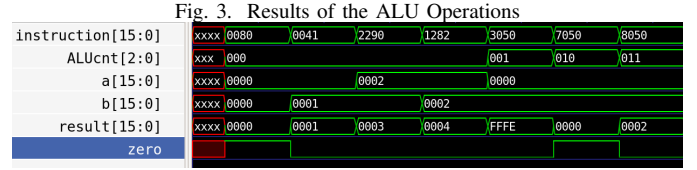


TABLE I
ALU FUNCTIONAL VERIFICATION TRACE

Time (ns)	ALUcnt	Op A	Op B	Result	Operation
0–10	000	000A	0005	000F	ADD
10–20	001	0007	0007	0000	SUB (Zero High)
30–40	010	1E15	F0FF	1015	AND (Masking)
60–70	101	1E15	0001	3C2A	SLL
90–100	111	0004	0008	0001	SLT (True)

B. Register and Memory Integration

Modular testing of the General Purpose Register (GPR) file confirmed stable synchronous write behavior, with data captured exclusively on the positive rising clock edge. Write-enable logic protections successfully prevented unintended register overwrites. Simultaneously, the dual-port asynchronous read architecture proved capable of retrieving and propagating two concurrent operands to the ALU input buses within a fraction of a single cycle, ensuring the ALU receives stable data well before the next clock edge.

Instruction Memory (ROM) validation demonstrated that the instruction fetch path operates correctly under the Harvard architecture paradigm. The ROM responds to Program Counter (PC) updates purely combinatorially. When the PC increments, the bit-slicing logic successfully translates the 16-bit address into a 4-bit word index, outputting the correct instruction instantaneously without requiring a clock synchronization phase.

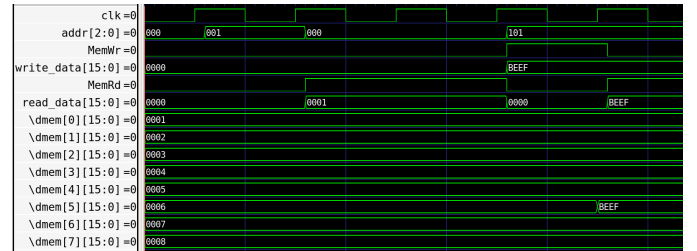


Fig. 4. Results of Data Memory Testing

C. System-Level Execution

Full-system integration testing utilised a program-driven stimulus via the test.prog initialisation file. The processor successfully executed complex multi-instruction sequences, including chained Load-Add-Store operations, as detailed in Table II.. Hierarchical signal monitoring of the datapath and Control Unit confirmed that:

Fig. 5. Results of System Integration Test



- Conditional branches correctly evaluate the ALU zero flag, appropriately multiplexing the PC input to either the incremented PC or the calculated branch target.
- The reserved opcode 1010 triggers a safe state, maintaining the current architectural state without causing unintended memory or register writes.
- Data memory (RAM) correctly stores ALU results following Store instructions and asynchronously provides read data during Load instructions without causing bus contention.

TABLE II
INTEGRATION TEST: EXPECTED VS. OBSERVED PROCESSOR STATE

Time	PC	Instruction (Hex)	Expected Hardware State	Observed GTKWave State	Status
10ns	0000	LD R2, 0(R0) (0080)	R2 ← dmem[0] (0001)	\registers[2] = 0001	PASS
30ns	0004	ADD R2, R1, R2 (2290)	R2 ← 0002 + 0001 = 0003	\registers[2] = 0003	PASS
40ns	0006	ST R2, 2(R1) (1282)	dmem[4] ← 0003	\dmem[4] = 0003	PASS
50ns	0008	SUB R2, R0, R1 (3050)	R2 ← 0 - 2 = FFFE	\registers[2] = FFFE	PASS
120ns	0016	JMP 0 (D000)	PC ← 0000 (Pipeline Reset)	pc updates to 0000	PASS

V. DISCUSSION

The results confirm that the StarCore-1 meets its primary design objective as a single-cycle RISC processor. The ALU is the most critical validation point, as timing violations here would propagate throughout the entire datapath.

A. Timing Integrity and Signal Stability

Waveform analysis confirms that all combinational paths — most critically the Instruction Memory fetch, Control Unit decoding, and ALU execution — settle well before the subsequent rising clock edge. All stages complete within the 10 ns clock period, validating operation at 100 MHz. Signal transitions in GTKWave are free of glitches, racing conditions, and unintended latches, confirming structural robustness essential for aerospace deployment.

B. Single-Cycle vs. Pipelined Design

The single-cycle model eliminates inter-stage pipeline registers, reducing area and dynamic power consumption — a meaningful advantage within the 400 mW OrbitEdge-1 power budget. However, the clock period is bounded by the worst-case critical path spanning fetch, decode, ALU execution, memory access, and write-back in a single combinational chain. A pipelined design would partition this path across stages, permitting a higher clock frequency at the cost of hazard detection logic and increased control complexity. The single-cycle approach therefore trades peak performance for determinism and simplicity, appropriate for pre-programmed telemetry workloads.

C. Branch and Jump Latency

Branch and jump resolution occurs entirely within the same cycle as fetch: the branch target is computed by the dedicated adder and the correct PC input is selected by the target multiplexer before the next rising edge. There is therefore no branch penalty, a direct benefit of the single-cycle model that eliminates the need for branch prediction or pipeline flush logic.

D. Co-Processor Integration

Integrating a co-processor via opcode 1010 would require three targeted modifications. First, the Control Unit must assert a new *CoProEn* signal when 1010 is decoded, while keeping all main-datapath signals de-asserted. Second, the *Mem2Reg* multiplexer must be extended to a 3-to-1 mux, adding the co-processor result bus as a third write-back source. Third, for write bus arbitration: mutual exclusivity is enforced by construction since opcode 1010 never activates the R-type control path, meaning only one result bus is ever driven per cycle. If the co-processor requires multiple cycles, a *done* flag must gate a clock-enable on the PC and all synchronous state elements, stalling the processor until the result is ready.

VI. CONCLUSION

The design, implementation, and verification of the StarCore-1 single-cycle microprocessor were successfully completed using Verilog HDL. By adopting a Harvard architecture, the processor eliminated structural memory hazards and achieved simultaneous instruction fetch and data access within a single clock period. Rigorous modular verification confirmed the functional correctness of individual components, including the purely combinational ALU and the strictly synchronous General Purpose Registers. Subsequent full-system integration testing proved that the unified datapath accurately executes complex instruction sequences, correctly evaluates conditional branches, and maintains timing constraints suitable for a 100 MHz operating frequency. The robust handling of edge cases and safe-state opcodes further demonstrates the processor's fault tolerance. Ultimately, the StarCore-1 has proven to be a highly efficient, deterministic, and reliable edge-computing core, fully satisfying the requirements for the OrbitEdge-1 aerospace mission.